



Mohalla.

Your neighbourhood, one tap away

How It Works & How to Present It

Part A: the architecture and every technical piece, explained simply.

Part B: a slide-by-slide plan to make the PPT engaging (and funny), the demo as a story, and tough viva questions with answers.

Architecture

Database

Security

Presenting

What's inside

Part A explains the project so you can answer any “how does it work?” question. Part B turns that into a presentation people actually enjoy. Read Part A once; rehearse Part B twice.

#	Section	Use it for
A1	The big picture (architecture diagram)	Slide 12 · “explain your architecture”
A2	Life of one search request	Explaining how all parts work together
A3	Frontend: React website + screens	Slides 4, 6 · frontend questions
A4	Backend: Express API and its 38 routes	Backend / API questions
A5	Database design (ER diagram)	Slide 13 · database questions
A6	Real data from OpenStreetMap	Slide 7 · “is the data real?”
A7	The 4 unique features under the hood	Slides 5, 6, 8, 9
A8	Security	“Are passwords safe?” and similar
A9	Android app	Slide 10
A10	Hosting, GitHub & deployment	“Is it live?” · cloud questions
A11	Testing, numbers & honest limitations	Slide 15 · tricky questions
A12	Jargon in one line each	Quick revision before the viva
B1	The 7 golden rules of an engaging talk	Mindset
B2	Three opening hooks	The first 30 seconds
B3	Slide-by-slide script with jokes (all 17 slides)	The actual talk
B4	The live demo as a story	The demo
B5	Humour rules and rescue lines	When a joke or the demo flops
B6	Voice, body language, team roles, rehearsal plan	Delivery
B7	Tough viva questions with model answers	Questions round

38 API routes	8 database tables	95 automated tests, all pass	~4,700 lines of app code	₹0 total cost
-------------------------	-----------------------------	--	------------------------------------	-------------------------

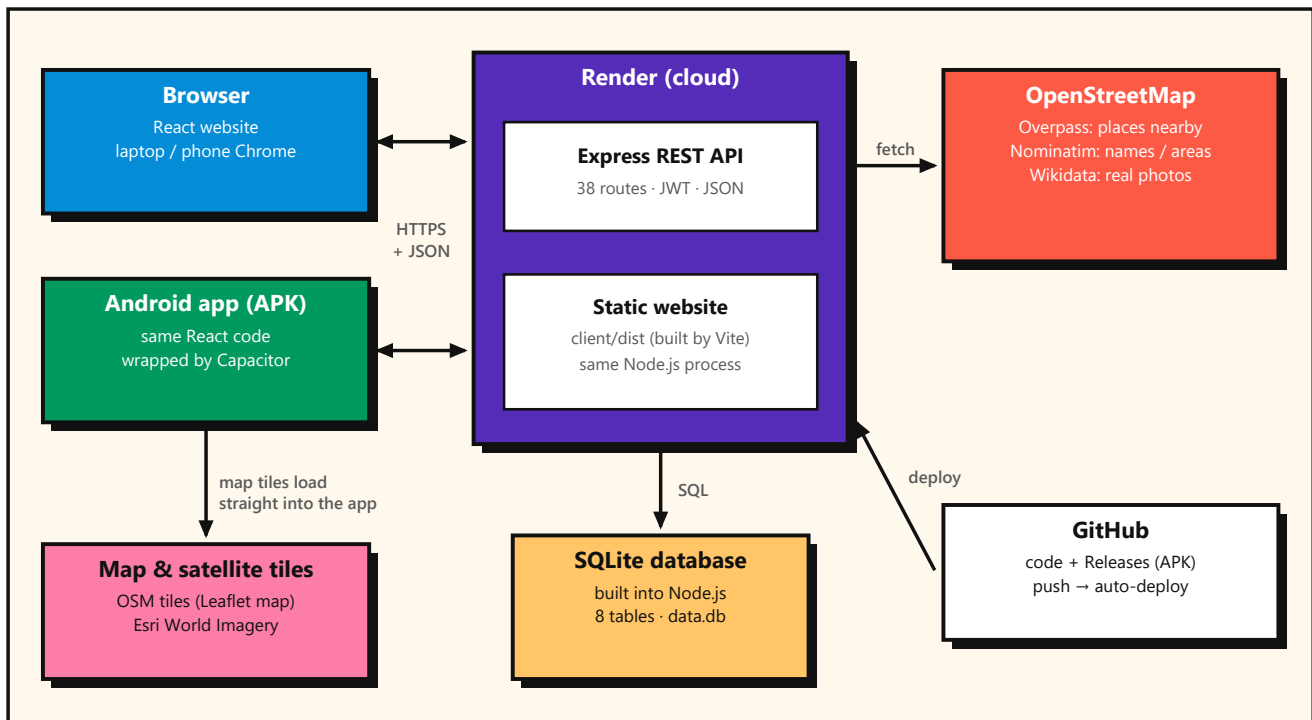
Part A

How Mohalla works

architecture · request flow · database · real data
security · Android · hosting · testing

A1 The big picture

Mohalla is a **client-server** app. The **clients** (the website in a browser and the Android app) only show screens. All data lives on **one server**, which answers requests through a **REST API** that speaks **JSON**. The same server also hands out the website itself, so one Node.js process runs everything.



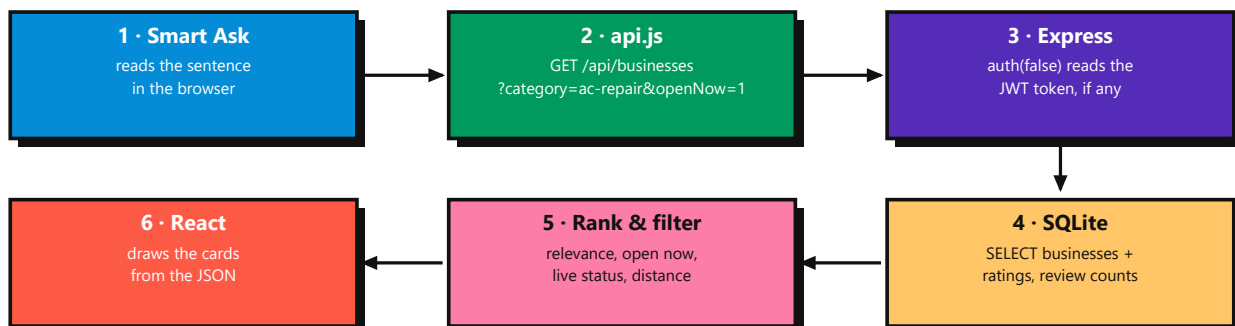
EXPLAIN IT LIKE A RESTAURANT

The **React app** is the waiter: it takes your order and brings the food. The **Express API** is the kitchen counter: it checks the order is valid and who you are. **SQLite** is the fridge where everything is stored.

OpenStreetMap is the vegetable market we go to when the fridge doesn't have something yet. And **Render** is the building we rent — for free, so it closes when nobody is eating.

A2 Life of one search request

Follow one search from typing to results — this is the best way to show you understand the whole system. The user types “my AC is not cooling, need someone urgently”:



Round trip: well under a second (plus up to ~1 min if the free Render server was asleep).

Step	What happens	Where in the code
1	Smart Ask spots “ac”, “not cooling” → category AC Repair, and “urgently” → Open now + nearest first. This runs in the browser, no server needed.	client/src/smartAsk.js

Step	What happens	Where in the code
2	The React page builds the URL and calls the API with fetch. If logged in, it adds the JWT token in the Authorization: Bearer ... header.	client/src/api.js
3	Express receives the request. The auth(false) middleware reads the token if there is one (search works logged-out too).	server/index.js
4	One SQL query fetches approved businesses with their category, average rating and review count.	BIZ_SELECT in index.js
5	JavaScript then scores relevance, works out open now from the timings, applies the owner's Live Status (ignored if older than 12 h), measures distance from your GPS, filters, sorts and pages the list.	relevance(), isOpenNow()
6	The JSON comes back and React draws the business cards, badges and the map.	pages/Search.jsx, BusinessCard.jsx

A3 Frontend — what the user sees

The frontend is a **single-page app (SPA)** written in **React 19** and built with **Vite**. The browser loads one page once; after that **React Router** swaps screens instantly without reloading.

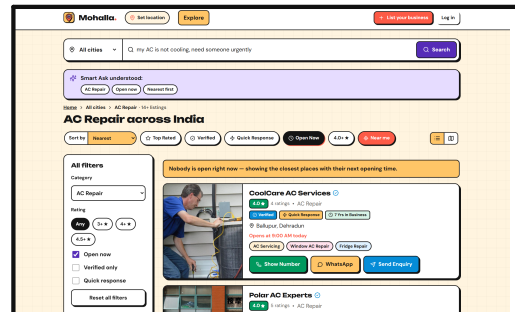
Screens (src/pages): Home, Search, BusinessDetail, Auth (login/register), Profile, Dashboard (owner), BusinessForm, Admin, Emergency, Credits, ServerSetup.

Reusable components (src/components): BusinessCard, SearchBar, MapView, EnquiryModal, EnquiryThread (chat), LeadForm, LocationPrompt, Stars, Toast.

Maps: Leaflet with OpenStreetMap tiles. The satellite picture on a card is made by working out which Esri satellite *tile* contains the business's latitude/longitude — no API key needed.

Location: the browser's navigator.geolocation (GPS on phones).

Design: our own CSS — retro flat style with thick black outlines and hard shadows, from a fixed colour palette.



A4 Backend — the Express API

The server is **Node.js + Express 5**. Every request passes through a chain of **middleware** (small functions that run in order) before reaching its route:

```
cors() // allow the Android app / other origins to call the API
express.json({ limit: '1mb' }) // read JSON bodies, reject huge ones
auth(true | false) // read the JWT → req.user (true = must be logged in)
allow('business', 'admin') // role check → 403 if not allowed
route handler // run SQL, return JSON
```

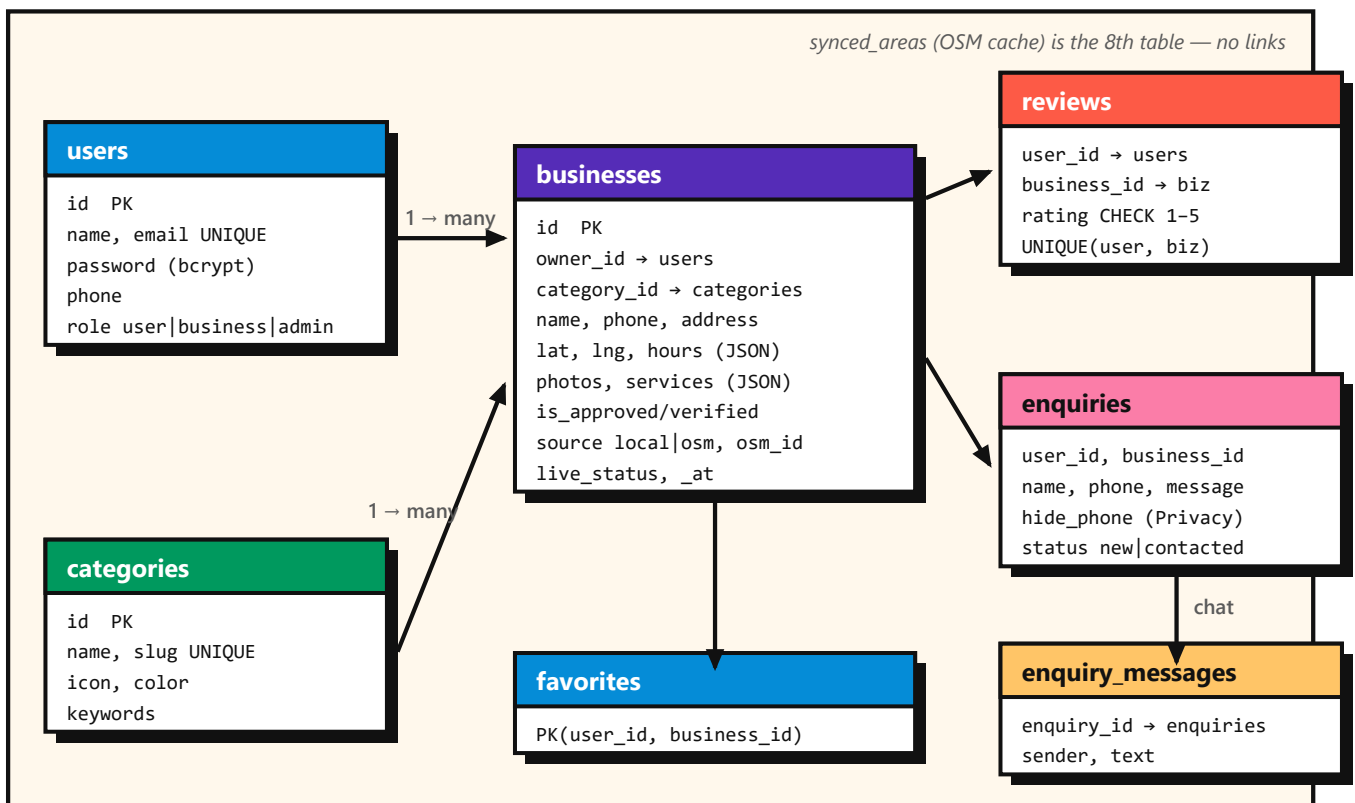
The 38 routes, grouped:

Group	Routes	Who
Auth	POST /api/auth/register · POST /api/auth/login · GET /api/auth/me	Anyone
Search & browse	GET /api/businesses · /api/businesses/:id · /api/suggest · /api/categories · /api/cities · /api/stats	Anyone
Real places	POST /api/nearby (import from OpenStreetMap) · POST /api/businesses/:id/claim	Anyone / owners
Listings	POST, PUT, DELETE /api/businesses · POST, DELETE .../photos · PATCH .../live-status	Owner of it, or admin
Reviews	POST /api/businesses/:id/reviews · DELETE /api/reviews/:id	Logged-in users
Enquiries & chat	POST .../enquiries · POST /api/leads · GET/POST /api/enquiries/:id/messages · PATCH /api/enquiries/:id	Customer & owner
Favourites & "my"	GET/POST/DELETE /api/favorites · GET /api/my/reviews, businesses, enquiries, sent-enquiries	Logged-in users
Admin	GET /api/admin/stats, businesses, users · PATCH businesses · DELETE users · POST/DELETE categories	Admin only

REST in one line: the URL says **what** (a business, a review), the HTTP verb says **what to do** (GET read, POST create, PUT/PATCH update, DELETE remove), and the status code says **how it went** (200 OK, 401 not logged in, 403 not allowed, 404 not found).

A5 Database design

We use **SQLite**, which is built into Node.js 22+ (node:sqlite). The whole database is one file, so there is no database server to install. Tables are created on start-up with CREATE TABLE IF NOT EXISTS, and new columns are added safely by a tiny migration helper (addColumn) — so old databases upgrade themselves.



Design choice	Why it matters
Foreign keys (e.g. reviews.business_id → businesses.id)	Links rows together; a review always belongs to a real user and business.
ON DELETE CASCADE	Delete a business and its reviews, enquiries and chat go with it — no orphan rows.
UNIQUE(user_id, business_id) on reviews	One review per person per business — stops rating-spam.
CHECK (rating BETWEEN 1 AND 5)	The database itself refuses a 6-star or 0-star rating.
Indexes on reviews, enquiries, owner, osm_id	Fast look-ups as data grows; the unique osm_id index stops importing the same real place twice.
JSON in TEXT columns (hours, photos, services)	Flexible lists without extra tables — fine at this size.
Prepared statements with ? placeholders	User input is never pasted into SQL → no SQL injection.

A6 Real data from OpenStreetMap

When you allow location, the app calls POST /api/nearby and the server's osm.js does this:

#	Step	Detail
1	Round your position to a ~1 km grid	e.g. 17.49, 78.40 — neighbours share one cache entry.
2	Check the cache	If this area was loaded in the last 24 h (synced_areas table), stop here — instant.
3	Ask the Overpass API	Everything with a name within 2 km that is a shop, clinic, ATM, restaurant... (up to 400). Three mirror servers are tried in turn if one is busy.
4	Translate tags → our categories	A TAG_MAP turns OSM tags like amenity=dentist into our doctors category, shop=hairdresser into Beauty & Spa, etc.
5	Read timings & services	Parses OSM opening_hours and tags like cuisine or wheelchair access.
6	Find a real photo	OSM image tag → Wikimedia Commons, or the Wikidata item's photo (P18), 50 at a time. No photo? The card shows a satellite view of the exact spot instead of a fake stock image.
7	Name the area	Nominatim reverse-geocodes the point (e.g. "KPHB Colony, Hyderabad").
8	Save	Upsert into businesses with source='osm'; the unique osm_id prevents duplicates. Owners can later claim them.

Searching a name that isn't saved yet (e.g. a college) also triggers a live Nominatim lookup. On the live server, one call near KPHB imported 400 real places in a few seconds.

A7 The 4 unique features — under the hood

Feature	How it actually works
Smart Ask	Rule-based language understanding (no paid AI). Word lists per category in English + Hinglish ("bijli", "dawai", "khana", "nal"...). Urgent words ("urgent", "abhi", "jaldi") → Open now + nearest. "near me", "paas" → sort by distance. "best", "acha" → Top Rated. If the text looks like a name, not a question, it searches the name instead. Works offline and costs ₹0.
Privacy Mode	The enquiry is saved with <code>hide_phone = 1</code> . When the owner's dashboard asks for enquiries, the server masks the number before sending it (only the last 2 digits show). Both sides talk through <code>enquiry_messages</code> — an in-app chat. The real number never leaves the server.
Live Status	Owner taps Available / Busy / Closed today → saved with a timestamp. When listings are read, a status older than 12 hours is ignored, so a forgotten "Available" can't mislead customers. "Closed today" also removes the business from Open-now results.
Emergency mode	No login needed. Gets your GPS, pulls the nearest hospitals, pharmacies and ATMs (real OSM data), sorts by distance, shows Call / Go buttons and the helplines 112, 108, 100, 101, 1091.
Bonus	Walking time next to nearby places; and if nothing is open, results fall back to the nearest places with their next opening time instead of an empty page.

A8 Security

Threat	What we did
Stolen database leaks passwords	Passwords are hashed with bcrypt (10 rounds, salted). We store the hash, never the password. Even we can't read them.
Someone pretends to be another user	Login returns a JWT signed with a secret key (valid 30 days). Changing even one character of the token breaks the signature, so the server rejects it. The secret lives in an environment variable on Render.
A customer opens the admin panel	Role checks on the server (<code>allow('admin')</code>), not just hidden buttons. Owners can only edit their own listings (<code>canEdit</code>).
SQL injection	Every query uses prepared statements with <code>?</code> placeholders.
Uploading a virus as "photo.jpg"	Only JPG/PNG/WEBP/GIF types, max 5 MB and 6 files, and we check the file's first bytes (its "magic number") to prove it's really an image. Files get random UUID names.
Huge requests to crash the server	JSON bodies are capped at 1 MB; list sizes are capped (max 50 per page).
Spam calls (our main promise)	Privacy Mode masks phone numbers on the server before they are sent anywhere.

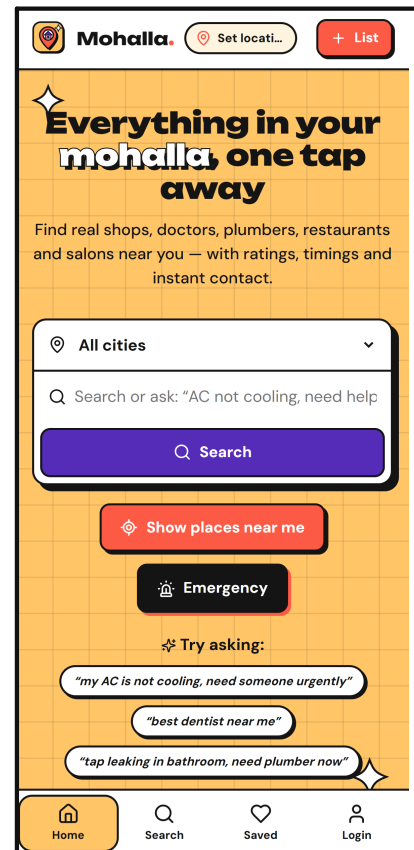
A9 The Android app

We didn't write a second app. **Capacitor 6** takes the same React code, builds it with `vite build --mode app` into `dist-app`, and packs it into a native Android project. Gradle then produces `Mohalla.apk`.

Inside the APK the screens are local files, so the app needs to know where the API is. That address is baked in at build time from `client/.env.app` (`VITE_API_URL=https://mohalla-b01x.onrender.com`).

On start, a **ServerGate** pings the server. Because the free Render server sleeps, it waits up to 75 seconds and shows “Waking up the server...”. If the server can't be reached, the **Connect to server** screen lets you use a laptop on the same Wi-Fi instead, or tap **Use online server** to switch back.

Phone features used: GPS location, tel: links for Call, WhatsApp links, and Google Maps directions.



A10 Hosting, GitHub & deployment

GitHub (private repo **ANSHUL-REAL/mohalla**) stores the code; big files (APK, PPT, videos) are on the **Releases** page. **Render** watches the repo: every push to main rebuilds and redeploys automatically — that's basic **CI/CD**.

```
git push // code goes to GitHub
npm run setup && npm run build // Render build: install packages, build the website
npm run seed && npm start // Render start: fresh demo data, start the server on $PORT
```

Render setting	Value / why
Plan	Free web service — sleeps after ~15 min idle, wakes in ~1 min on the next visit.
NODE_VERSION	24 — the version we tested on (built-in SQLite needs 22.13+).
JWT_SECRET	Generated by Render, kept out of the code.
Disk	Temporary on the free plan: the database resets on every restart, so the start command re-seeds demo data. For real users we would add a persistent disk or PostgreSQL.
HTTPS	Render gives a free certificate, so everything is encrypted in transit.

A11 Testing, numbers & honest limitations

Automated tests: 95 tests in `server/test/api.test.mjs` using Node's built-in test runner. They start a fresh server with a separate test database and call every route: login, permissions, search filters, reviews, Privacy Mode masking, uploads, admin actions and bad input. A load test fires many searches in parallel. Stress-testing found ~20 bugs (crashes on bad input, an unsafe upload); all were fixed. All 95 pass, 0 errors under load.

16 categories	1,200+ demo reviews	400 real places per area import	12 h live-status expiry
-------------------------	-------------------------------	---	-----------------------------------

Honest limitations — say these before the examiner does, it scores points:

Limitation	Why it's OK for now / the fix
Free Render server sleeps and resets its data	Fine for a demo. Real launch: paid instance + persistent database.
SQLite allows one writer at a time	Perfect for thousands of users. For lakhs: PostgreSQL — the SQL barely changes.
Smart Ask is keyword rules, not real AI	Fast, free, offline and predictable. Next step: an AI model for harder sentences.
OpenStreetMap is missing some Indian shops	That's why owners can add and claim listings — the data improves with use.
Public Overpass servers can be slow	We cache each area for 24 h and fall back across three mirrors.

A12 Jargon in one line each

Term	Meaning in our project
API / REST	The menu of URLs the apps call to get or change data (<code>GET /api/businesses</code>).
JSON	The text format the data travels in: <code>{"name": "CoolCare", "rating": 4.6}</code> .
SPA	Single-page app: the site loads once, then React swaps screens without reloading.
Middleware	A function every request passes through, e.g. checking the login token.
JWT	A signed login “pass” the server gives you; you show it with every request.
Hashing (bcrypt)	One-way scrambling of passwords; you can check a password but never reverse it.
CORS	Browser rule about which websites may call an API; we allow the Android app to call ours.
Foreign key / cascade	A link from one table's row to another's; cascade deletes linked rows together.
Geolocation / geocoding	Getting your GPS position / turning coordinates into a place name (Nominatim).
Capacitor	Tool that packs a web app into a real Android app.
CI/CD	Push code → it is built and deployed automatically (GitHub → Render).
Cold start	The first request after the free server slept; it takes ~1 minute to wake up.

Part B

How to present it

hooks · slide-by-slide script with jokes

demo as a story · humour rules · tough viva questions

B1**The 7 golden rules of an engaging talk**

#	Rule	In practice
1	Tell a story, not a spec sheet	Problem → frustration → our fix → proof (demo). People remember stories, not tech stacks.
2	One idea per slide	If a slide needs 2 minutes of reading, it's a document, not a slide. Talk; don't read.
3	Make them do something early	A show of hands in the first minute wakes up the room (see Hook B).
4	Humour = relatable truth	Spam calls, "Open 24×7" shops that are shut, aunties who know everything. Not random memes.
5	Show, don't claim	Never say "it's fast" — click it. Never say "it's secure" — show the masked number.
6	Use callbacks	Bring back an early joke at the end (the spam call). It makes the talk feel planned and complete.
7	Know your first and last line by heart	Everything in between can be natural. Start strong, end strong.

B2**Three opening hooks — pick one****HOOK A · THE FAKE SPAM CALL (most fun, needs a friend)**

Ask a friend in the audience to call your phone right as you begin. Answer on speaker: **"Hello sir, you searched for a plumber in 2022, would you like a credit card?"** Hang up, look at the audience: **"That... is exactly why we built Mohalla."** Then start slide 1.

HOOK B · THE SHOW OF HANDS (zero risk)

"Raise your hand if you've ever searched for a plumber, electrician or tuition online... Keep it up if you then got calls for the next six months... Keep it up if one of them was selling insurance."
(Hands stay up, people laugh.) **"Good — you are our target audience."**

HOOK C · THE LAUNCH VIDEO (most polished)

Play **Mohalla_Launch.mp4** (48 s) full screen with sound before saying anything. When it ends, pause one second, then: **"Hi, we're the people who made that. Let us show you it's real."**

Combine B + C: hands first (people engage), then the video (people are impressed).

B3**Slide-by-slide script with jokes**

For each of the 17 slides: what the slide is for, a line to say, and an optional joke or interaction. Jokes are marked **FUN**; use about one every 2–3 slides, not on every slide. Your speaker notes in the PPT have the factual script; this adds the energy.

Slide 1 · Title — Mohalla · about 20 s

Purpose: Introduce the name: "Mohalla" means neighbourhood.

FUN "Every mohalla already has a JustDial — it's called that one aunty who knows every shop, every plumber and everyone's business. We just built the app version. Without the gossip."

Slide 2 · The problem · about 45 s

Purpose: The four pains: spam calls, wrong timings, exact category names, missing places.

FUN *Do Hook B here if you didn't open with it. For timings: "Every shop board says Open 24×7. You go at 9 pm — shutter down, dog sleeping outside." For categories: "Nobody types HVAC services. We type AC kharab hai."*

Slide 3 · Our solution · about 25 s

Purpose: One app: search + real places + safe contact.

FUN *"Think of it as JustDial... that doesn't give your number to the whole city."*

Slide 4 · Core features · about 30 s

Purpose: The standard directory features — move fast, the demo will show them.

FUN *"These are the features every directory app must have. Our job was to make sure they don't break during this demo. Fingers crossed." (Cross your fingers.)*

Slide 5 · What makes us different · about 30 s

Purpose: The 4 unique features. Slow down here — this is your scoring slide.

FUN *"If you remember only one slide today, make it this one. If you remember none, at least remember we were nice."*

Slide 6 · Smart Ask · about 45 s

Purpose: Normal sentences → filters, in English and Hinglish.

FUN *"Smart Ask understands Hinglish better than my relatives understand what I study." Interaction: ask the audience for a problem and type it live (test a few the day before).*

Slide 7 · Real places (OpenStreetMap) · about 40 s

Purpose: Live, real data; real photos or satellite view.

FUN *"We did not type 400 shops by hand. We're students, not interns." Then show your college: "And yes — we literally put our college on the map."*

Slide 8 · Privacy Mode & Live Status · about 45 s

Purpose: Masked numbers + in-app chat; live status that expires after 12 h.

FUN *"The business sees xxxxxxxx03. That's it. Your number is now safer than your Instagram password." Live Status: "It expires in 12 hours — so unlike that shop board, it can't lie for long."*

Slide 9 · Emergency mode · about 30 s

Purpose: Nearest hospitals, pharmacies, ATMs + helplines.

FUN **No jokes on this slide.** *Lower your voice, slow down: "When someone is hurt, nobody wants to scroll. One tap." The change in tone makes it hit harder — and shows maturity.*

Slide 10 · Android app · about 30 s

Purpose: Same code, two platforms, via Capacitor.

FUN *"We wrote the code once. Writing it twice is how you lose a semester." Hold up your phone with the app open.*

Slide 11 · User roles · about 25 s

Purpose: Customer, owner, admin; admin approval keeps fake listings out.

FUN *"The admin is basically the class monitor. Approves listings so no fake Best Biryani in Hyderabad goes live... although honestly, every biryani place in Hyderabad claims that."*

Slide 12 · Architecture · about 45 s

Purpose: Client-server: apps → Express API → SQLite; OSM for real data.

FUN *Use the restaurant analogy from A1: waiter, kitchen, fridge, vegetable market. Point at each box as you say it. Examiners love a clear analogy.*

Slide 13 · Database design · about 40 s

Purpose: Tables, foreign keys, one review per user per business.

FUN *"Our tables are like a family WhatsApp group — everyone is connected, and if you delete one person, a lot of messages disappear with them. In databases we call that ON DELETE CASCADE."*

Slide 14 · Tech stack · about 25 s

Purpose: React, Node, Express, SQLite, Capacitor, OSM — all free and open source.

FUN *"Total infrastructure cost of this project: zero rupees. Total chai cost: please don't ask."*

Slide 15 · Testing · about 35 s

Purpose: 95 automated tests, load test, ~20 bugs found and fixed.

FUN *"Our stress test found 20 bugs. We fixed all 20. We're not saying there are zero bugs left — we're saying we haven't found them yet."*

Slide 16 · Future scope · about 30 s

Purpose: UPI booking, notifications, voice search, languages, OTP reviews, Play Store.

FUN *"And the Play Store launch — as soon as our parents approve the developer-account fee."*

Slide 17 · Thank you · about 15 s

Purpose: Thank the panel, invite questions, then go to the demo.

FUN Callback to the opening: *"And if you get a spam call after this presentation — it wasn't us. We have Privacy Mode."*

B4 The live demo as a story

Don't demo features — demo a **day in someone's life**. Give the user a name and a problem, and every click becomes the next part of the story. Example cast:

Scene	Story line	What you click
1	"Meet Rahul. Hostel in KPHB. It's May. His AC just died."	Home page → allow location → real places load
2	"Rahul doesn't know the word <i>HVAC</i> . He types like a human."	Smart Ask: <i>my AC is not cooling, need someone urgently</i>
3	"He wants someone good, who's actually open."	Top Rated chip → open CoolCare → reviews, timings, map
4	"But Rahul has trust issues with sharing his number. Fair."	Log in as user → Send Enquiry with Privacy Mode
5	"Meanwhile, at CoolCare..."	Log in as owner → masked number → reply in chat → set Available now
6	"And someone has to keep the fake listings out."	Admin → approve FitZone → stats
7	"Two hours later, Rahul's friend twists an ankle on the stairs."	Emergency mode (serious tone)
8	"And Rahul? He does all of this from his phone."	Show the Android app on your phone

IF THE RENDER SERVER IS WAKING UP

"Our server is on the free plan, so — like every college student at 8 am — it takes a minute to wake up." Use that minute: explain the architecture slide while it loads. Better still, open the site 2 minutes before you start so it's already awake.

B5 Humour rules and rescue lines

Do	Don't
Joke about shared, everyday pain (spam calls, shop boards, Hyderabad heat)	Joke about teachers, classmates, religion, politics or anyone in the room
Joke about yourselves ("we're students, not interns")	Insult JustDial or other apps — compare features, not people
Pause after the punchline; smile; let them laugh	Explain the joke or laugh at your own joke first
Keep it to ~6–8 light moments in 12 minutes	Put a joke on Emergency mode or on the security slide
Say jokes in your own words — natural beats perfect	Read a joke off the slide

Rescue lines (memorise two):

Situation	Say
A joke gets silence	"That joke was in Privacy Mode — only some of you received it."
The demo freezes	"This is the part where we prove the 95 tests were necessary." Then refresh, or play Mohalla_Demo.mp4.

Situation	Say
Internet is down	"The real-places feature needs internet — the rest runs on our own server." Switch to the laptop server or the demo video.
You forget your line	Look at the slide, take a breath, and say "The key point here is..." — nobody notices.
A question you can't answer	"Good question — I don't want to guess. What I can tell you is..." and answer the nearest thing you do know.

B6**Voice, body language, team roles, rehearsal**

Area	Tips
Voice	Slower than feels natural. Pause before key numbers ("... <i>ninety-five</i> tests"). Drop your voice on Emergency mode.
Eyes	Look at the examiners, not the screen. Glance at the slide only to point at something.
Hands	Point at diagram boxes as you name them. Hold up the phone for the Android slide. Hands out of pockets.
Clicker	Use a presenter remote or a phone app so you can move around instead of hiding behind the laptop.
Team of 2–4	Split by role: Storyteller (slides 1–9), Demo driver (live demo), Tech lead (slides 10–16 + hard questions). Hand over with a line: "...and to show you it's real, here's Priya."
Rehearsal plan	Day –2: read Part A, run the demo once. Day –1: full run with a timer twice (aim 12 min), test your Smart Ask sentences, record yourself once on the phone. Day 0: open the live site 2 min early, run the checklist in the Start & Demo Guide.

B7**Tough viva questions with model answers**

Question	Model answer
Why JWT and not sessions?	JWT is stateless: the server doesn't store sessions, it just verifies the signature. That fits a REST API used by both a website and a mobile app. Trade-off: a token can't be cancelled before it expires (30 days), so for production we'd add shorter tokens with refresh tokens.
How do you stop SQL injection?	All queries are prepared statements with ? placeholders, so user input is sent as data, never as part of the SQL command.
What if two people review the same business twice?	The database has UNIQUE(user_id, business_id) on reviews, so a second review from the same user can't be added; posting again <i>updates</i> their existing review (an upsert). Owners can't review their own business, and the CHECK constraint keeps ratings between 1 and 5.
Why Capacitor instead of React Native or Kotlin?	We already had a full React web app. Capacitor reuses 100% of that code, so one codebase gives both web and Android. React Native would mean rewriting every screen.
Why SQLite? Will it scale?	Zero setup and built into Node — great for a project and for thousands of users. It allows one writer at a time, so for very high traffic we'd move to PostgreSQL; our SQL is standard so the change is small.

Question	Model answer
Is the data real?	Our own listings are demo data (clearly marked). Real places come live from OpenStreetMap — you can turn on location anywhere and see real shops around you, with their real photos or satellite view.
Is Smart Ask AI?	It's rule-based natural-language processing: keyword lists in English and Hinglish mapped to categories and filters. It's instant, free and works offline. A machine-learning model is in our future scope.
How is the phone number actually hidden?	The enquiry is stored with <code>hide_phone = 1</code> . When the owner fetches enquiries, the server masks the number before sending the response, so the full number never reaches the owner's device.
What does CORS do in your app?	Browsers block a page from calling an API on a different address unless the API allows it. The Android app runs from a local address, so the API sends CORS headers allowing it.
Why does the online data reset?	Render's free plan has a temporary disk, so we re-seed demo data on every start. With a paid persistent disk or a managed database, data would stay.
What happens when the free server is asleep?	The first request wakes it in about a minute. The Android app waits up to 75 s and shows "Waking up the server..." instead of failing.
What was the hardest part?	Pick a true one, e.g. mapping messy OpenStreetMap tags to clean categories and getting real photos, or making the same code work offline-bundled in the APK and online on the web.

LAST LINE OF THE DAY

"So Mohalla does everything a directory app should, and adds real live data, no-spam privacy, live status, Smart Ask and emergency help — built for ₹0 and running live right now. Thank you."